Visualisierung

Das Training Schritt für Schritt

Inhaltsverzeichnis

Bedienung 1
Was die Bereiche zeigen 1
Wie die Seite entsteht 2

Eine eigenständige Webseite zeigt, wie die Netze lernen: vom zufälligen Start bis zum Ende der 20 000 Epochen. Sie besteht aus einer einzigen HTML-Datei ohne externe Bibliotheken und läuft deshalb auch offline im Browser.

Bedienung

- **Szenario wählen:** XOR mit Seed 0, XOR mit Seed 8 (bleibt im lokalen Minimum hängen) oder der 7-Segment-Decoder mit Seed 42. Das gewählte Szenario steht in der Adresse, z. B. `#xor-stuck`; ein solcher Link öffnet direkt dieses Szenario.
- **Durch das Training gehen:** Der Schieberegler wählt einen Zeitpunkt, ▸ spielt das Training ab.
- **Sample wählen:** Ein Klick auf eine Zeile der Tabelle „Vorhersagen„, auf einen Punkt der Entscheidungsfläche oder auf eine Ziffer zeigt im Netzgraphen die Aktivierungen für genau diesen Input.
- **Genaue Werte:** Der Mauszeiger über einer Linie, einem Neuron oder einem Segment zeigt den Zahlenwert.

Was die Bereiche zeigen

Bereich	Inhalt
Kennzahlen	Epoche, mittlere Kosten J und wie viele Samples richtig gerundet werden
Netz	Neuronen als Kreise, die Füllung ist die Aktivierung beim gewählten Sample; Linien sind Gewichte, blau positiv, rot negativ, die Linienstärke entspricht dem Betrag
Kostenverlauf	mittlere Kosten über alle Epochen mit logarithmischer y-Achse; der Punkt markiert den gewählten Zeitpunkt
Entscheidungsfläche (XOR)	Output des Netzes für 25 × 25 Punkte im Quadrat $[0, 1]^2$, die Grenze liegt bei 0,5; die vier Trainingspunkte sind markiert, falsch vorhergesagte rot umrandet

Bereich	Inhalt
7-Segment-Anzeige	die Helligkeit jedes Segments ist der Output des zugehörigen Neurons; falsche Segmente sind rot umrandet
Vorhersagen	alle Samples mit Soll, Ausgabe und gerundetem Ergebnis

Am XOR-Szenario mit Seed 0 lohnt sich der Abschnitt zwischen etwa Epoche 4 000 und 6 000: Dort fallen die Kosten stark, und Gewichte wie Entscheidungsfläche ändern sich am deutlichsten. Beim Seed 8 enden $[1, 0]$ und $[1, 1]$ beide bei etwa 0,5.

Wie die Seite entsteht

```
flowchart LR
    T["python -m neural_network visualize"] --> F["fit() mit on_epoch<br>3 Szenarien"]
    F --> S["Schnappschüsse<br>Gewichte, Aktivierungen, Kosten"]
    S --> J["JSON"]
    J --> H["template.html<br>HTML, CSS, JavaScript"]
    H --> I["visualization/index.html"]
```

- `python -m neural_network visualize` trainiert alle drei Szenarien mit der Python-Version (rund 20 Sekunden) und schreibt die Seite nach `visualization/index.html`.
- Während des Trainings ruft `fit` nach jeder Epoche die Funktion `on_epoch` auf. An ausgewählten Epochen wird ein **Schnappschuss** gespeichert: alle Gewichte und Biases, die Aktivierungen jedes Neurons für jedes Sample, die mittleren Kosten und beim XOR zusätzlich das Raster der Entscheidungsfläche. Die Aktivierungen kosten nichts extra, weil sich jedes Neuron seinen letzten Output für die Backpropagation ohnehin merkt.
- Die Zeitpunkte sind **logarithmisch verteilt**: 79 Schnappschüsse, am Anfang dicht, später weit auseinander, weil sich das Netz am Anfang am stärksten ändert. Die Kostenkurve hat 227 Punkte, ebenfalls logarithmisch verteilt.
- Gewichte werden auf 4, Aktivierungen auf 3 Nachkommastellen gerundet; die fertige Seite ist so etwa 900 KB groß.
- Die Daten werden als JSON in `template.html` eingesetzt. Die Seite zeichnet Netz und Kostenkurve als SVG und die Entscheidungsfläche als Canvas, hell oder dunkel nach Systemeinstellung.
- **Die Seite rechnet selbst nichts.** Jeder angezeigte Wert stammt aus dem echten Training; was man sieht, ist genau das Netz aus der Implementierung.

Beim Veröffentlichen kopiert der Website-Build die Seite als `visualisierung/` mit. Ändert sich das Netz oder das Training, muss sie vorher neu erzeugt werden. Die C#-Version hat keine eigene Visualisierung; sie dient dem Nachvollziehen im Debugger.